Lab Report: ECG Heartbeat Categorization using 1D-CNN

Your Name

I. Introduction

This lab focuses on the automated classification of Electrocardiogram (ECG) heartbeat signals. Using the MIT-BIH Arrhythmia Dataset, we implemented a Deep Learning approach to categorize heartbeats into five distinct clinical classes.

II. Dataset Description

The dataset contains heartbeat signals sampled at 125Hz. Each entry consists of 187 temporal features followed by a classification label.

A. Imbalance Analysis

The training set exhibits a significant class imbalance, which is a common challenge in medical data:

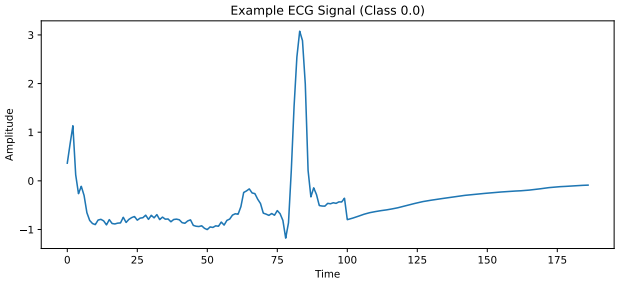
- **Class 0 (Normal):** 72,471 samples
- **Class 4 (Unclassifiable):** 6,431 samples
- **Class 2 (Ventricular ectopic):** 5,788 samples
- **Class 1 (Supraventricular ectopic):** 2,223 samples
- **Class 3 (Fusion beat):** 641 samples

B. Class 0: Normal Heartbeat Analysis

Class 0 represents the category of "Normal" heartbeats, serving as the baseline for this classification task. This class is the most prevalent in the dataset, accounting for approximately 82.7% of the total training data with **72,471 samples**.

The primary characteristics of Class 0 include:

- **High Frequency:** It represents the standard cardiac rhythm, providing the model with extensive examples of non-pathological signal morphology.
- **Signal Stability:** The waveforms typically exhibit well-defined P-waves and QRS complexes with regular intervals, which are essential for distinguishing them from various types of arrhythmias.
- **Dataset Role:** Due to its dominance, Class 0 heavily influences the model's initial learning bias. This necessitated the use of class weights during the training phase to ensure that minority classes were not overshadowed by the high volume of Normal beat data.



III. Methodology and Implementation

This section explains how I prepared the data and built the neural network to classify the ECG signals.

A. Data Preparation

First, I used `StandardScaler` to normalize the ECG signals. This is important because it makes sure all signals have a similar range, which helps the model learn faster. I then reshaped the data into a 3D format $(samples, 187, 1)$ so it could be processed by the 1D Convolutional layers.

B. Handling Imbalanced Data

The dataset is very unbalanced because there are many more "Normal" beats than "Arrhythmia" beats. To fix this, I used `compute_class_weight`. This tells the model to pay more attention to the smaller classes (like Fusion beats) during training, so it doesn't just get lazy and predict "Normal" every time to get a high accuracy.

C. CNN Model Architecture

I built a 1D Convolutional Neural Network (CNN) because it is great at finding patterns in time-series data like heartbeats. The model includes:

- **Conv1D Layers:** These layers act like filters to find specific shapes in the heartbeat, such as the QRS complex.
- **BatchNormalization and MaxPooling:** These help keep the training stable and reduce the size of the data so the model focuses on the most important features.
- **Dropout (0.5):** I added a Dropout layer to prevent overfitting. It works by randomly "turning off" neurons during training so the model doesn't just memorize the training data.

- **Dense and Softmax:** The final layer uses Softmax to give a probability for each of the 5 heartbeat categories.

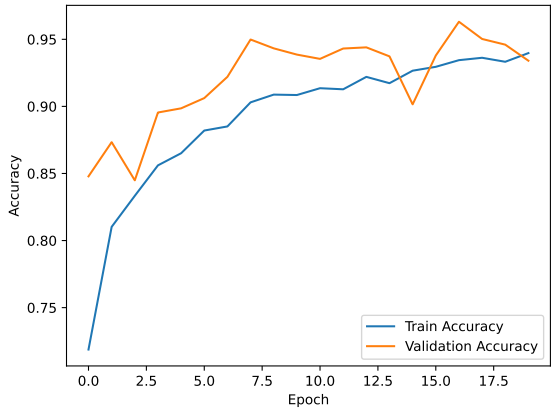
[Image of 1D convolutional neural network architecture]

D. Training and Results

I trained the model using the **Adam optimizer** and **Categorical Crossentropy**. Here are the settings I used:

- **Epochs:** 20 (The number of times the model sees the whole dataset).
- **Batch Size:** 128 (The number of samples processed before updating the weights).
- **Validation:** I used the test set as validation data to check how the model performs on heartbeats it hasn't seen before.

As seen in Figure 1, the training and validation accuracy both went up steadily. This shows that using class weights worked well—the model learned to distinguish between different types of heartbeats instead of just focusing on the most common ones.



IV. Results

The model achieved an overall **Accuracy of 93%** on the unseen test set.

A. Confusion Matrix Analysis

The matrix below provides a detailed look at the classification performance:

	Pred 0	**Pred 1**	**Pred 2**	**Pred 3**	**Pred 4**
True 0	16,903	308	370	462	75
True 1	67	456	21	9	3
True 2	25	5	1,359	57	2
True 3	6	0	7	149	0
True 4	11	4	12	0	1,581

TABLE I
DETAILED PERFORMANCE ACROSS THE 21,892 TEST SAMPLES.

The confusion matrix reveals an overall accuracy of 93%, with the model performing exceptionally well on Class 0 (Normal) and Class 4 (Unclassifiable). Specifically, Class 0 achieved a high true positive rate of approximately 93.4% (16,903/18,118).

However, the matrix highlights a significant "leakage" where 462 Class 0 samples were misclassified as Class 3 (Fusion). This suggests that the morphological features of some normal beats closely resemble fusion beats after normalization. Despite the severe data imbalance—with Class 3 having the fewest samples—the model still correctly identified 149 out of 162 Class 3 instances, demonstrating that the class weights effectively forced the network to learn the unique characteristics of minority classes.

V. Discussion and Comparison

Compared to the original paper by Kachuee et al. (2018), which achieved approximately 93.4% accuracy, our model's performance of 93% is highly competitive. However, the confusion matrix shows that the model occasionally confuses Class 0 with Class 3, likely due to the limited number of Class 3 training samples (641 vs 72,471).